

Fintech Project:

This project is a secure fintech-style authentication and session management backend system that I developed for a US freelance client.

The main goal of the project was to build an enterprise-level authentication architecture focused on security, session handling, fraud prevention, and scalable user management.

I built the backend using Node.js, Express.js, MongoDB, and Redis. The system includes advanced security mechanisms like JWT authentication, refresh token rotation, OTP-based login verification, CSRF protection, Redis-based session management, rate limiting, and secure cookie handling.

In this project, I implemented:

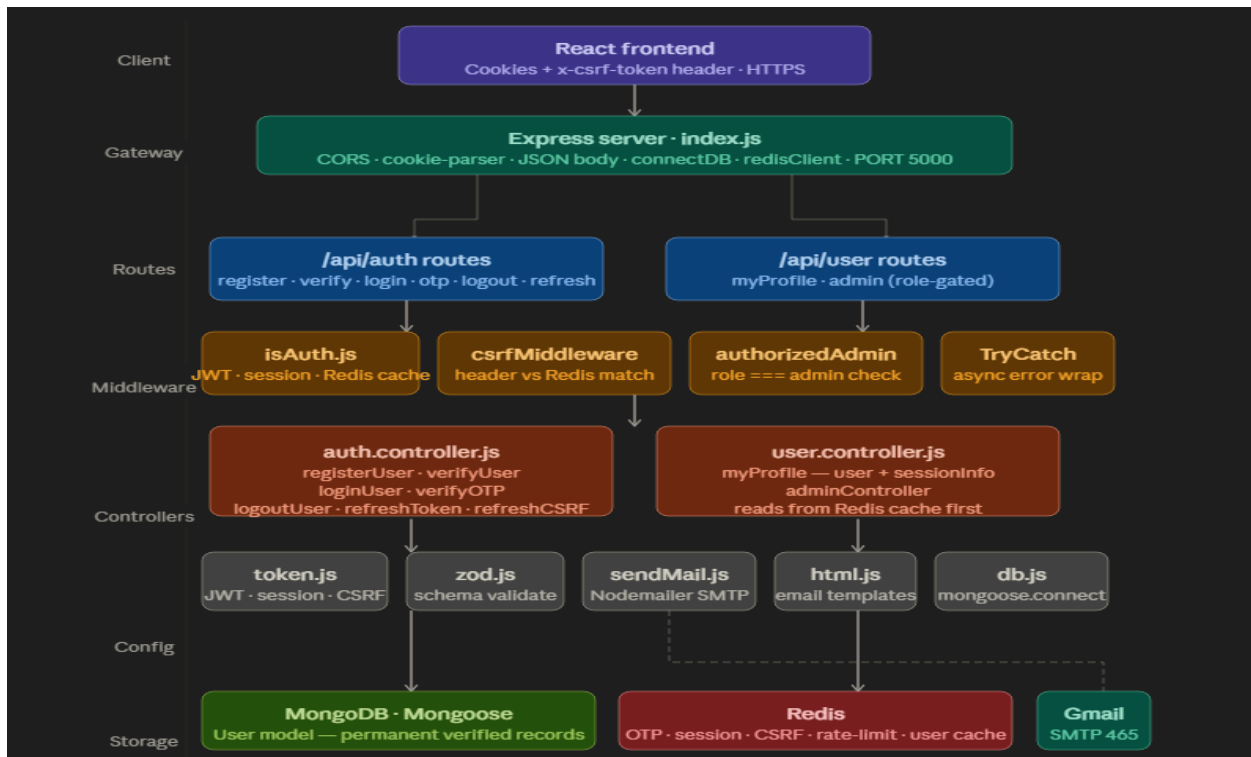
- User registration with email verification
- OTP-based secure login system
- JWT access and refresh token architecture
- Redis session storage and single-device session handling
- CSRF token generation and validation
- Role-based access control for admin and users
- Rate limiting to prevent brute-force attacks
- Secure password hashing using bcrypt
- Redis caching for temporary authentication data

The architecture was designed similar to modern fintech applications where security, session integrity, and scalability are critical.

I was mainly responsible for backend architecture design, authentication flow, Redis session handling, API development, token management, and implementing security best practices.

1. Problem Statement

In fintech applications, security is the highest priority because sensitive user data and financial sessions must be protected from attacks like session hijacking, CSRF attacks, brute-force login attempts, and token misuse.



So the client wanted a secure authentication system with scalable session management and strong security practices

1. Entry Point — index.js

This file bootstraps the entire backend server.

Flow

```

dotenv.config()
↓
await connectDB()
↓
Redis Initialization
↓
Global Middlewares
↓
CORS Configuration
↓
Route Mounting
↓

```

```
app.listen()
```

What I have Implemented

Environment Configuration

```
dotenv.config()
```

Loads secure environment variables.

MongoDB Initialization

```
await connectDB()
```

Important Design Choice

Server DOES NOT start if MongoDB fails.

Redis Initialization

```
export const redisClient = createClient({  
  url: redisUrl  
})
```

I have exported a singleton Redis instance.

Redis Failure Handling

```
redisClient.on('error', ...)
```

Redis failure is non-fatal.

Backend still runs.

Why?

MongoDB = Critical dependency
Redis = Performance dependency

CORS Setup

credentials: true

Allows cookies across frontend/backend domains.

Routes Mounted

/api/auth

/api/user

2. User Model — user.model.js

Defines MongoDB user structure.

Schema Design

name

email

password

role

timestamps

Key Security Design

Passwords are already hashed BEFORE reaching the model.

No pre-save hooks.

Role-Based Access Control

enum: ["user", "admin"]

3. Routes Layer

auth.routes.js

Endpoint	Purpose
POST /register	Register user
GET /verify/:token	Verify email
POST /login	Login
POST /verify-otp	OTP verification
POST /logout	Logout
POST /refresh	Refresh JWT
POST /refresh-csrf	Refresh CSRF

user.routes.js

Endpoint	Middleware Chain
GET /profile	isAuth → csrf → myProfile
GET /admin	isAuth → authorizedAdmin

Design Principle

Routes are THIN.

No business logic.

Only:

Middleware wiring
Controller mapping

Note: I kept the routes layer intentionally thin so business logic remains inside controllers and middleware. This improves maintainability and testing.

4. Middleware Layer : isAuth.js

This is the CORE SECURITY LAYER.

Authentication Flow

Read JWT Cookie



Verify JWT



Validate Active Session in Redis



Check Redis User Cache



Fallback Mongo Query



Attach req.user

JWT Verification

`jwt.verify()`

Verifies:

Token integrity
Signature
Expiration

Session Validation

`isSessionActive()`

This checks:

`active_session:{userId}`

inside Redis.

Enterprise Feature

Single Device Login Enforcement

JWTs are normally stateless, meaning a valid token works until expiry. I added Redis-backed session validation on every request so tokens can be invalidated immediately by removing the session from Redis. This enabled single-session enforcement similar to fintech applications.

Redis User Cache

`user:{userId}`

Avoids MongoDB queries repeatedly.

Benefits

- ✓ Faster APIs
- ✓ Reduced DB load
- ✓ Better scalability

Token Expiry Handling

`TokenExpiredError`

Returns:

401 → frontend refreshes token

Admin Authorization

authorizedAdmin

Checks:

```
req.user.role === "admin"
```

5. CSRF Middleware : csrfMiddleware.js

This is ADVANCED SECURITY.

Purpose

Protect against:

Cross Site Request Forgery (CSRF)

Flow

Frontend sends x-csrf-token header

↓

Server fetches csrf:{userId} from Redis

↓

Exact token comparison

↓

Allow or reject

GET Is Skipped

GET requests are read-only.

CSRF mostly targets:

POST
PUT
DELETE

Token Generation

`crypto.randomBytes(32)`

Redis Storage

`csrf:{userId}`
TTL: 1 hour

A CSRF attacker can automatically send cookies but cannot read the CSRF token from JavaScript. The frontend reads the CSRF token from the cookie and attaches it as a custom header, which the backend validates against Redis.

6. TryCatch Middleware

Pattern Used

Higher Order Function
Decorator Pattern

Flow

Controller
↓
Wrapped in TryCatch
↓
Errors auto-forwarded

Benefits

- ✓ No repeated try/catch
- ✓ Cleaner controllers
- ✓ Centralized error handling

TryCatch wraps all async controllers and automatically forwards errors to Express error handling middleware. This prevents repetitive try/catch blocks in every controller.

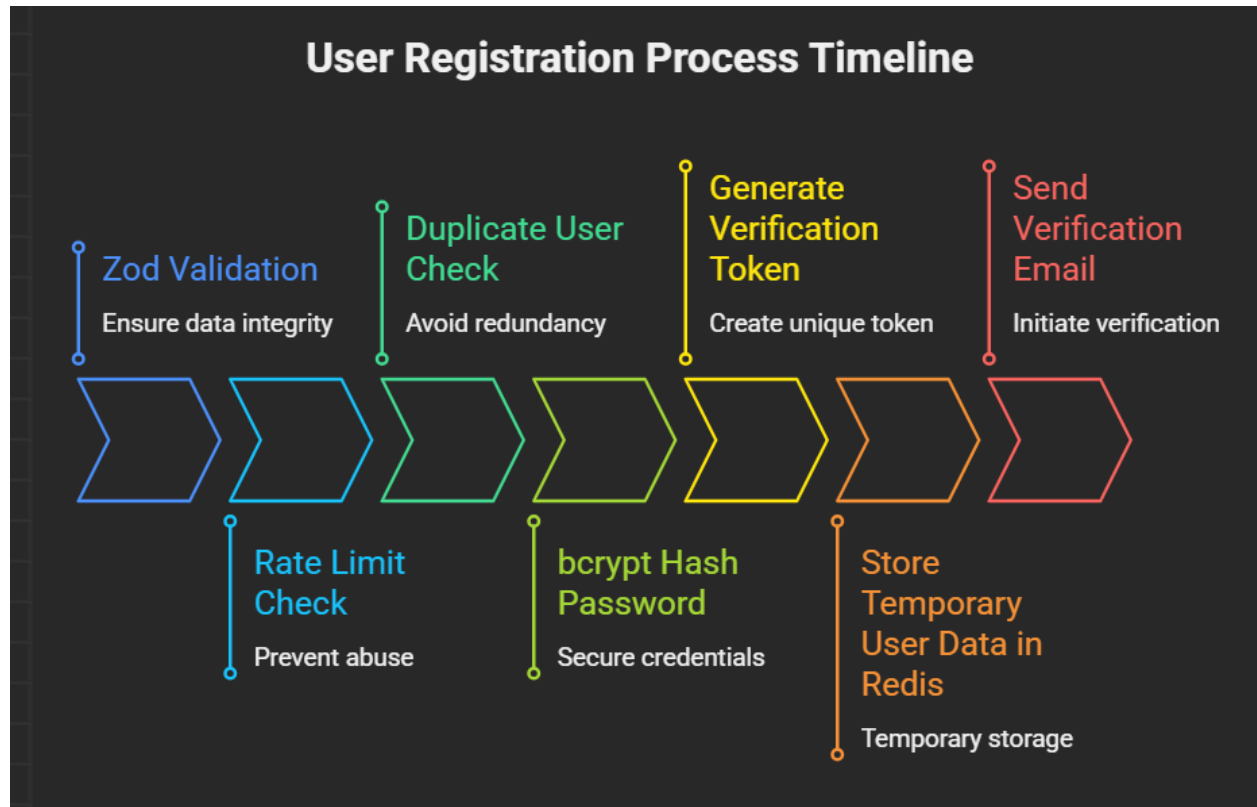
7. Auth Controller — `auth.controller.js`

This is the MAIN BUSINESS LOGIC layer.

`registerUser()`

Flow

Rate limiting of same Ip+same Email:



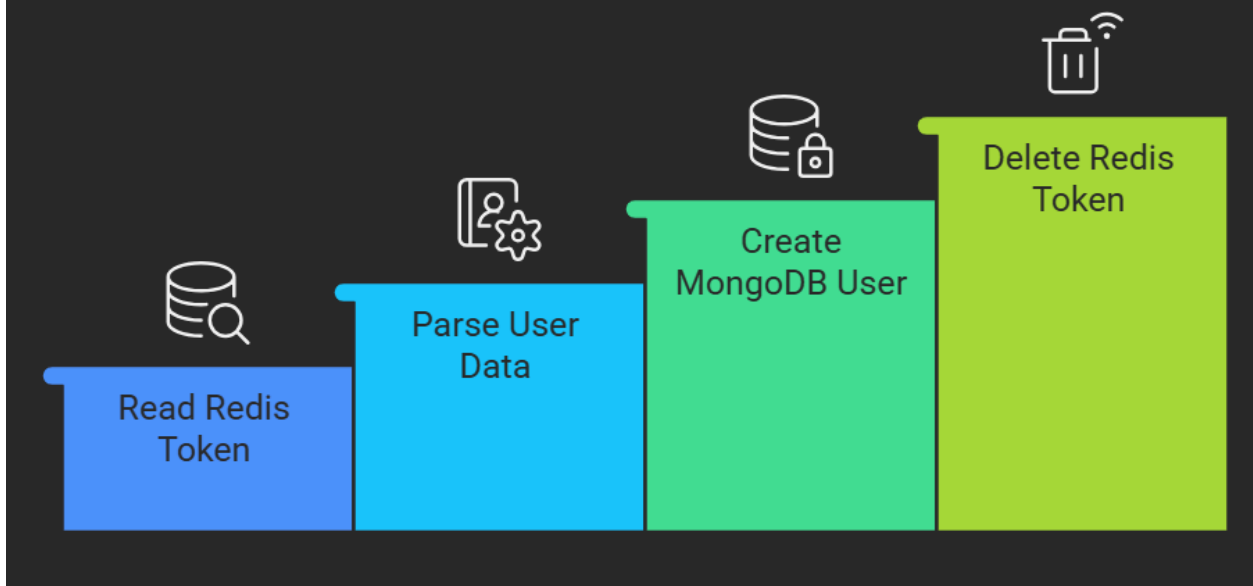
Redis Key

verify:{token}
TTL: 5 minutes

During registration I stored temporary user data inside Redis until email verification succeeds. This prevents unverified or fake users from polluting the database.

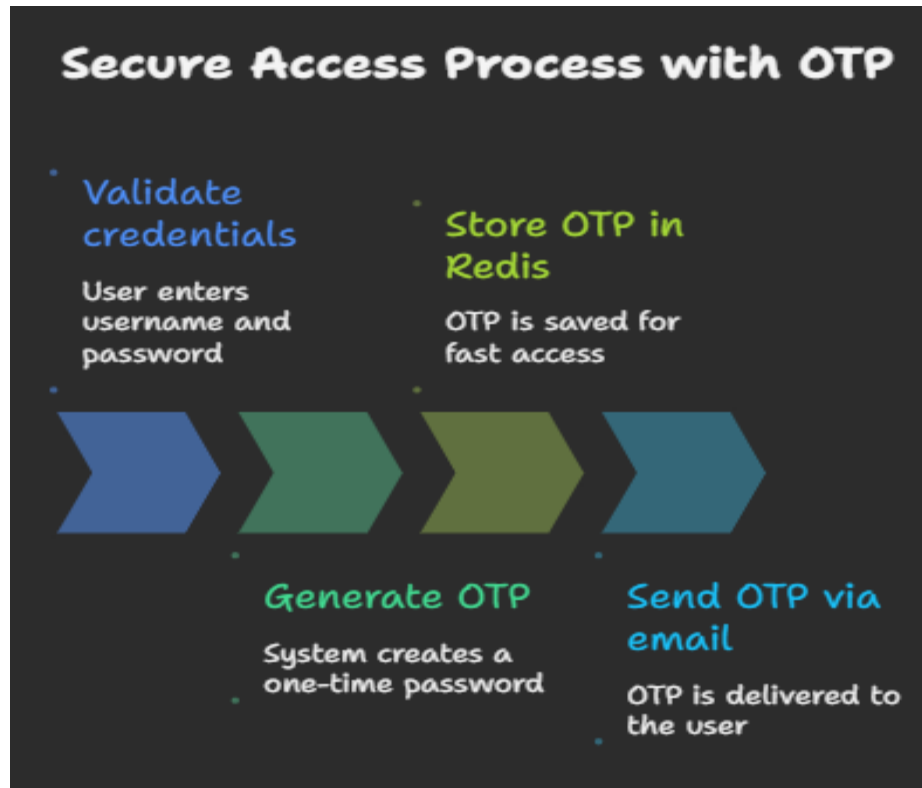
verifyUser()

User Authentication Process



loginUser()

Flow



OTP Design

6-digit OTP

TTL: 5 minutes

Security Feature

2-Factor Authentication

The login flow uses password verification plus OTP verification. Even if a password is compromised, the attacker still needs access to the registered email to complete login.

verifyOTP()

Flow

Verify OTP



Delete OTP



Generate Access + Refresh Tokens



Create Redis Session



Generate CSRF Token



Send Cookies

logoutUser()

What Happens

Delete refresh token

Delete session

Delete CSRF token

Delete user cache

Clear cookies

refreshToken()

Flow

Verify Refresh Token



Generate New Access Token



Set New Cookie

8. user.controller.js

myProfile()

Flow

Get req.user



Get sessionId



Fetch Redis session



Return session metadata

Important Optimization

No MongoDB query on cache hit.

Session Info Returned

loginTime

lastActivity

sessionId

adminController()

Protected by:

isAuth

authorizedAdmin

9. Token Lifecycle — token.js

This file controls COMPLETE SESSION MANAGEMENT.

generateToken()

Steps

Generate sessionId



Generate Access Token



Generate Refresh Token



Delete old session



Store new Redis session

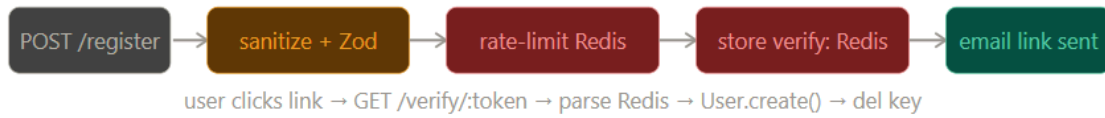


Set secure cookies



Generate CSRF token

Registration flow

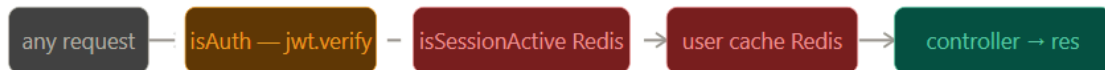


Login flow (2-factor)



user hits OTP → POST /verify-otp → compare Redis OTP → del key → generateToken() → 3 cookies set → return user + csrfToken

Authenticated request flow

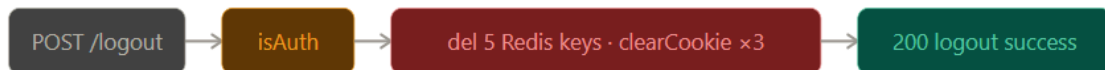


Express.js runs csrfMiddleware — reads x-csrf-token header → matches csrf:{userId} in Redis → 403 or next()

Token refresh



Logout flow



Redis key reference

verify:{token}	5 min · pending registration data	refresh_token:{id}	7d · raw JWT
otp(email)	5 min · login OTP	active_session:{id}	7d · current sessionId
*-rate-limit:{ip}:{email}	60 s · spam prevention	session:{sessionId}	7d · metadata + lastActivity
csrf:{userId}	1 hr · CSRF token	user:{userId}	1 hr · user object cache

MongoDB stores only one thing: verified User documents · everything else is Redis

Redis Keys Written

```

refresh_token:{userId}
active_session:{userId}
session:{sessionId}
    
```

Session Object

```

{
  "userId": "...",
    
```

```
"sessionId": "...",  
"createdAt": "...",  
"lastActivity": "..."  
}
```

verifyRefreshToken()

Checks

- JWT validity
- Refresh token exact match
- Active session validity
- Session existence

Enterprise-Level Security

Refresh token rotation with Redis validation.

Refresh tokens are validated against Redis on every refresh request. If a user logs in from another device, the old refresh token is overwritten in Redis, making previous sessions invalid immediately even if the JWT itself has not expired.

10. Database Layer — db.js

Connect MongoDB.

Design Choice

Fast failure strategy

If DB fails:

```
process.exit(1)
```

Why?

No point serving requests without database.

11. Email Layer — sendMail.js

Purpose

Send:

Verification emails
OTP emails

Transport

smtp.gmail.com
Port 465 SSL

Why Port 465

Encrypted from connection start.

For production-scale systems I would migrate to AWS SES or SendGrid for higher reliability and throughput, but Gmail SMTP was sufficient for the freelance project scale.

12. Validation Layer — zod.js

Validation Rules

- ✓ Min 8 chars
- ✓ Uppercase
- ✓ Lowercase

- ✓ Number
- ✓ Special character

Important Design

sanitize → validate

Why?

Prevent NoSQL injection first,
then validate structure.

safeParse()

Returns structured validation errors.

I used Zod safeParse instead of parse so validation failures return structured field-level errors instead of throwing exceptions.

13. Email Templates — html.js

Functions

Function	Purpose
getOtpHtml	OTP email
getVerifyEmailHtml	Verification link

14. Complete Redis Key Architecture

Redis Key	Purpose	TTL
-----------	---------	-----

verify:{token}	Temporary registration data	5 min
register-rate-limit:{ip}:{email}	Register spam prevention	60 sec
otp{email}	OTP storage	5 min
login-rate-limit:{ip}:{email}	OTP spam prevention	60 sec
refresh_token:{userId}	Refresh token validation	7 days
active_session:{userId}	Single session enforcement	7 days
session:{sessionId}	Session metadata	7 days
csrf:{userId}	CSRF validation	1 hr
user:{userId}	User cache	1 hr

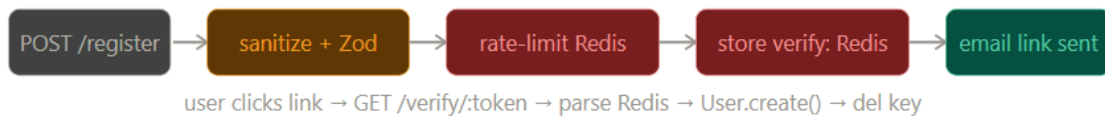
Redis Design Principle

Anything temporary or session-scoped lives in Redis.

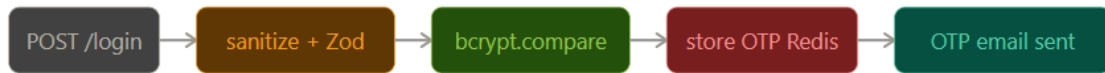
Permanent verified user data lives in MongoDB.

All llds:

Registration flow

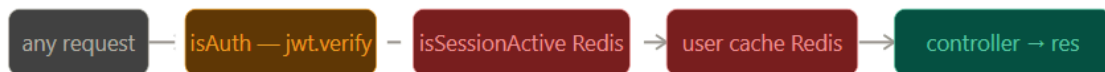


Login flow (2-factor)



hits OTP → POST /verify-otp → compare Redis OTP → del key → generateToken() → 3 cookies set → return user + csrfToken

Authenticated request flow

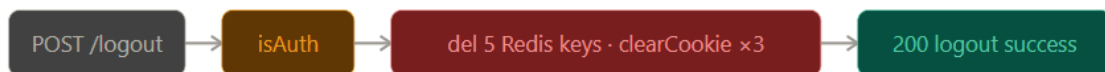


Express.js runs csrfMiddleware — reads x-csrf-token header → matches csrf:{userId} in Redis → 403 or next()

Token refresh



Logout flow



Redis key reference

verify:{token}	5 min · pending registration data	refresh_token:{id}	7d · raw JWT
otp(email)	5 min · login OTP	active_session:{id}	7d · current sessionId
*-rate-limit:{ip}:{email}	60 s · spam prevention	session:{sessionId}	7d · metadata + lastActivity
csrf:{userId}	1 hr · CSRF token	user:{userId}	1 hr · user object cache

MongoDB stores only one thing: verified User documents · everything else is Redis